

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«АЛТАЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных технологий

Кафедра информатики

**Разработка backend-части информационной системы
управления мероприятиями**
выпускная квалификационная работа

Выполнил:
студент группы 4.205-2,
Шацких Алексей Евгеньевич

(подпись)

Научный руководитель:
к.т.н., доцент
Михеева Татьяна Викторовна

(подпись)

Допустить к защите:
Зав. кафедрой, к.ф.-м.н., доцент
Козлов Денис Юрьевич

(подпись)

«__» _____ 2026 г.

Работа защищена:
«__» _____ 2026 г.

Оценка: _____

Председатель ГЭК:
д.т.н., профессор
Леонов Сергей Леонидович

(подпись)

Барнаул 2026

РЕФЕРАТ

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ BACKEND РАЗРАБОТКИ КЛИЕНТ- СЕРВЕРНЫХ ПРИЛОЖЕНИЙ	6
1.1. Методологии и подходы backend-разработки	6
1.2. Архитектурные подходы backend разработки	14
1.3. Анализ и выбор программных средств разработки	26
2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ	35
2.1. Расчёт годового экономического эффекта	35
2.2. Алгоритм текстового поиска	38
2.3. Алгоритм рекомендаций	40
2.4. Проектирование базы данных	42
3. РАЗРАБОТКА BACKEND-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ	45
3.1. Разработка доменного слоя	45
3.2. Разработка инфраструктурного слоя	55
3.3. Разработка слоя бизнес-логики	55
3.4. Разработка слоя API	55
ЗАКЛЮЧЕНИЕ	56
БИБЛИОГРАФИЧЕСКИЙ СПИСОК	57
ПРИЛОЖЕНИЕ	62

ВВЕДЕНИЕ

Мероприятия различного вида проводятся каждый день в миллионах организаций по всему миру. Для их проведения организаторам необходимо выбрать место, создать анонсы в мессенджерах и социальных сетях, узнать, какое количество человек планирует прийти и, возможно, создать таблицу для записи участников. Учитывая, что таких мероприятий в крупных организациях проводится большое количество, у организаторов возникают трудности в информационной поддержке организационных процессов для того, чтобы мероприятие состоялось.

Для решения этих проблем мы предлагаем создать информационную систему, позволяющую контролировать все процессы подготовки проведения мероприятий в одном месте с возможностью интеграции в уже существующую экосистему организации.

Актуальность темы обусловлена растущей цифровизацией всех отраслей экономики, что влечёт за собой увеличение потребности рынка в новых цифровых решениях привычных и рутинных задач.

Разработка предлагаемой информационной системы требует применения современных подходов и технологий, таких как использование паттернов проектирования, фреймворков, технологий тестирования и баз данных.

Данная выпускная квалификационная работа посвящена разработке backend-части системы, которая является ключевой составляющей всей системы, обеспечивая её стабильное функционирование, обработку бизнес-логики, хранение данных и интеграцию с внешними сервисами.

Целью данной выпускной квалификационной работы является разработка backend-части информационной управления мероприятиями.

Для достижения поставленной цели необходимо решить следующие **задачи:**

1. Изучить основные методы и технологии backend-разработки.
2. Выбрать архитектурные принципы и паттерны проектирования.
3. Выбрать программные средства разработки.
4. Спроектировать базу данных.
5. Разработать доменный слой.
6. Разработать инфраструктурный слой.
7. Разработать слой бизнес-логики.
8. Разработать слой API.

Объектом исследования данной работы является методология и практика разработки клиент-серверных приложений.

Предмет исследования – backend разработка с применением фреймворка ASP.NET.

Практическая значимость разработки backend-части системы управления мероприятиями заключается в создании поддерживаемой, расширяемой, масштабируемой и отказоустойчивой инфраструктуры системы.

Разрабатываемая система позволит упростить информационную поддержку процессов проведения мероприятий, что позволит увеличить вовлечённость участников.

Апробация. Основные результаты выпускной квалификационной работы докладывались на секции «Информационные системы и технологии» в рамках XIII региональной молодёжной конференции «Мой выбор – наука!» (г. Барнаул, 2026 г.).

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ BACKEND РАЗРАБОТКИ КЛИЕНТ-СЕРВЕРНЫХ ПРИЛОЖЕНИЙ

1.1. Методологии и подходы backend-разработки

При разработке backend-приложений, как и при разработке любого программного обеспечения, применяются различные методологии и подходы, позволяющие упростить разработку, поддержку, уменьшить количество ошибок в программном коде, а также увеличить взаимопонимание между заказчиком и командой разработки. Рассмотрим некоторые из них.

Test-Driven Development (TDD) [42] – методология, основополагающий принцип которого заключается в разработке программного обеспечения через тестирование. Процесс разработки по данной методологии происходит через небольшие итерации, проходящие в три этапа (рис. 1).

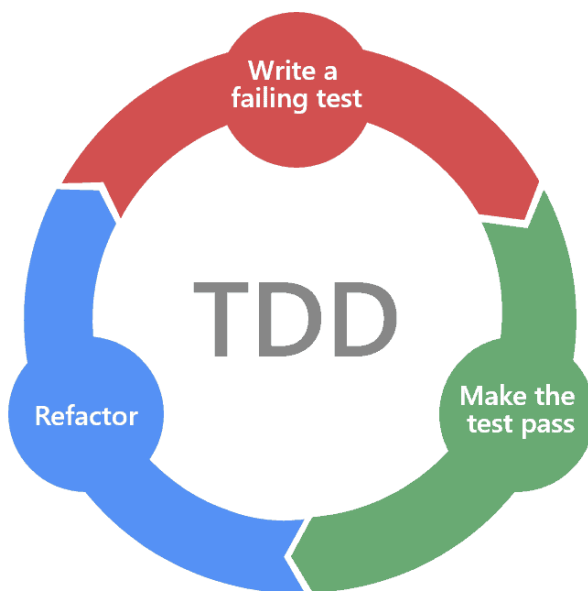


Рис. 1. Test-Driven Development

Первым этапом, именуемым «красной зоной», является написание тестов для ещё не существующего функционала, которые будут выдавать ошибку до того момента, пока не будет написана реализация.

Вторым этапом, именуемым «зелёной зоной», является написание реализации функционала, который будет проходить тесты без ошибок. Данный этап самый короткий и, если функционал проходит тесты, то он считается реализованным.

Третьим этапом, именуемым «синей зоной», является рефакторинг, то есть изменение и улучшение уже существующего функционала и тестов. В данной методологии рефакторинг является безопасным процессом, поскольку ошибки, возникающие при тестировании, содержат в себе точную информацию о месте и причине возникновения.

К достоинствам методологии относится создание тестируемого и легко поддерживаемого кода: поскольку тесты пишутся до реализации, разработчик вынужден проектировать модули с минимальной связностью, что улучшает архитектуру системы. Непрерывно пополняемый набор регрессионных тестов позволяет обнаруживать нарушения существующей функциональности сразу после внесения изменений. Помимо этого, тесты выполняют роль живой документации, точно описывая ожидаемое поведение каждого компонента.

К недостаткам относится замедление разработки на начальном этапе, пока команда не выработала устойчивый навык работы по данной методологии. Дополнительную сложность представляет тестирование кода, взаимодействующего с внешними системами, что требует применения макетов (mock-объектов). При нестабильных требованиях тесты нередко нуждаются в переработке наравне с реализацией, что может нивелировать часть выигрыша от раннего обнаружения ошибок.

Domain-Driven Design (DDD) [35] – это подход проектирования объектов программного обеспечения как сущностей реального мира, с их

бизнес-логикой и жизненным циклом, отсюда проистекает определение данного подхода как «предметно-ориентированное программирование». Рассмотрим основные правила, которые предлагает данный подход.

Первым правилом, которое предлагает DDD, является разделение объектов предметной области на сущности и объекты-значения.

Объекты-значения (value-objects) – это маленькие составные блоки предметной области, которые не являются уникальными, не имеют своей бизнес-логики, жизненного цикла и могут сравниваться по значению. Объекты-значения предполагаются неизменяемыми, поэтому вместо изменения уже существующего объекта класса предлагается создавать новый объект. Рекомендуется, чтобы объекты-значения ссылались только на другие объекты-значения посредством объектов классов.

Сущности (entities) – это объекты предметной области, которые составлены из объектов-значений, они имеют свою бизнес-логику, жизненный цикл и являются уникальными, поэтому для их однозначного определения вводится идентификатор, который становится операндом сравнения. Сущности предполагаются изменяемыми, но рекомендуется инкапсулировать логику их изменения в публичных методах, не допуская прямого изменения полей объекта класса. Также рекомендуется, чтобы сущности ссылались на объекты-значения и другие сущности посредством объектов классов.

Пример применения сущностей и объектов-значений представлен на рисунке 2.

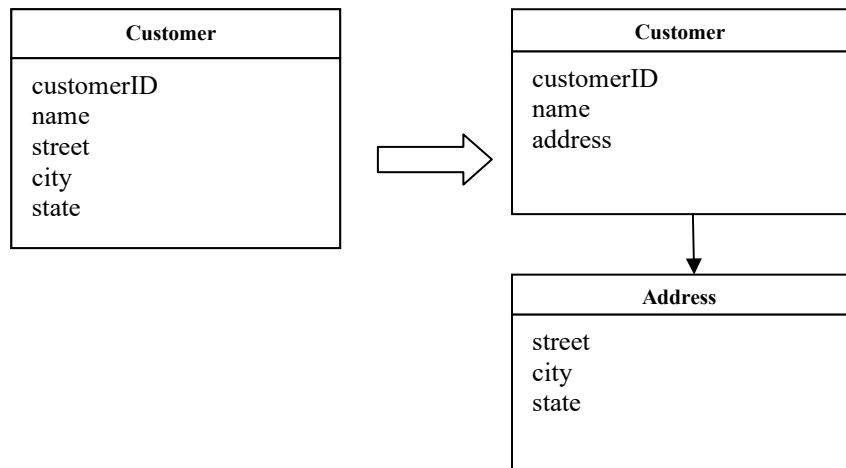


Рис. 2. Пример применения сущностей и объектов-значений

Вторым правилом, которое предлагает DDD, является выделение связанных между собой сущностей и объектов-значений в отдельные кластеры, называемые агрегатами. В каждом агрегате предлагается выделить главенствующую сущность, называемую корнем агрегата (*aggregate root*), которая будет отвечать за управление связанными сущностями и объектами-значениями (рис. 3). Рекомендуется, чтобы корень агрегата ссылался на другие корни агрегатов через идентификаторы.

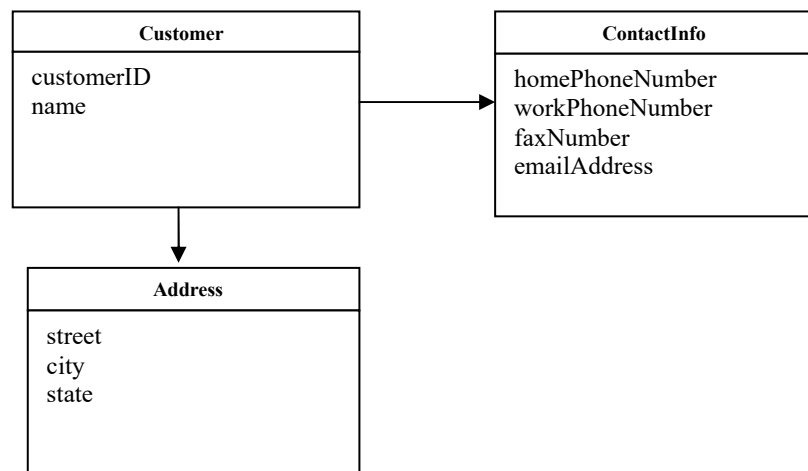


Рис. 3. Пример агрегата Customer

Третьим правилом, которое предлагает DDD, является выделение логики создания связанных сущностей в отдельные фабрики, что позволяет решить проблему разграничения зон ответственности сущностей (рис. 4).

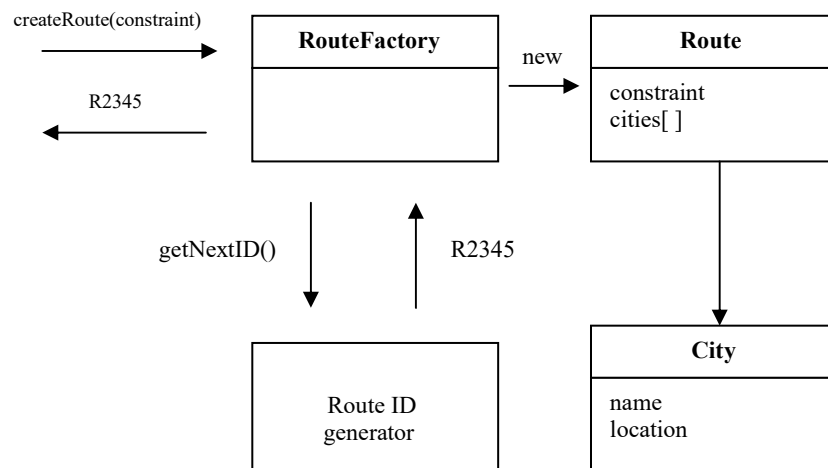


Рис. 4. Пример фабрики Route

Четвёртым правилом, которое предлагает DDD, является привнесение в доменную область репозитория (repositories) – объектов, которые отвечают за сохранение сущности в программном обеспечении (рис. 5). Как правило, в доменной области определяется только интерфейс репозитория, а непосредственно реализация этого интерфейса происходит в инфраструктурном слое приложения.

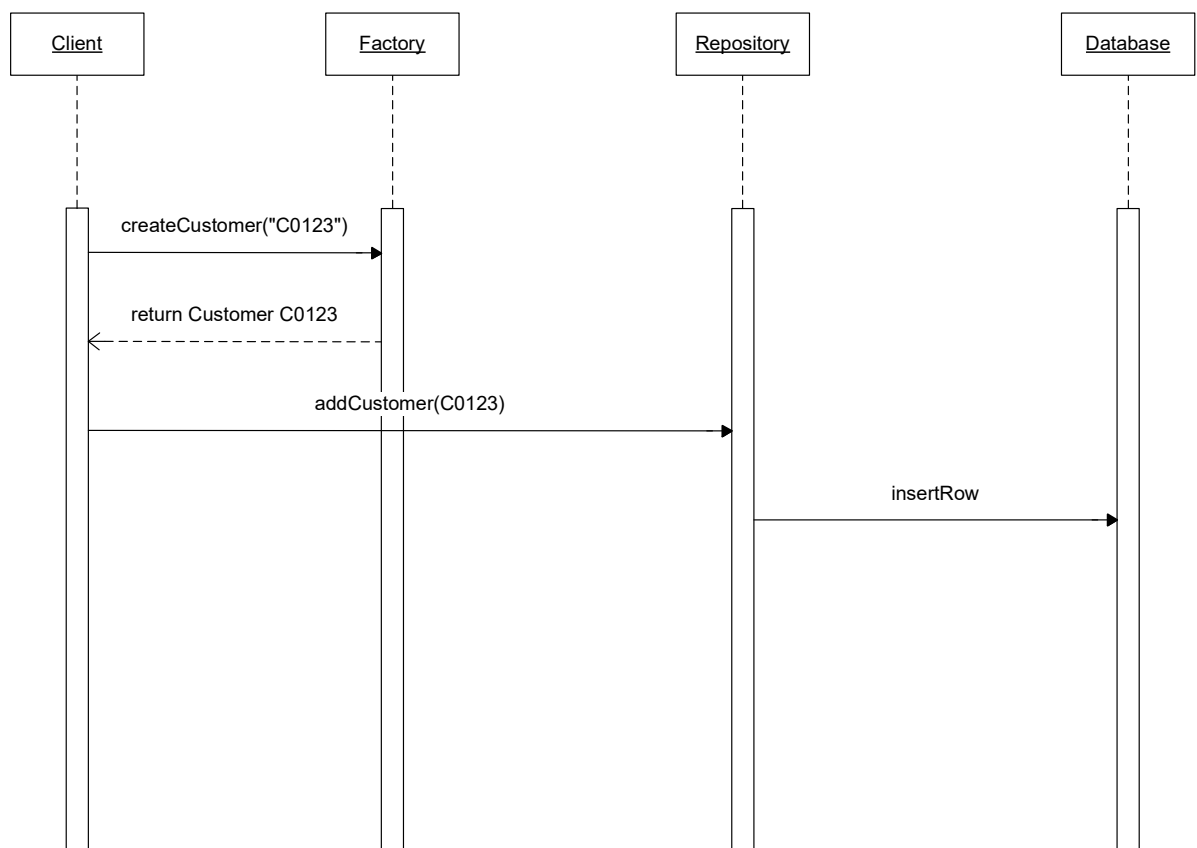


Рис. 5. Пример логики работы с репозиторием Customer

К достоинствам данного подхода относится инкапсуляция бизнес-логики в рамках единой доменной модели, что упрощает её повторное использование и перенос в между проектами, упрощение проектирования для разработчиков, не являющихся экспертами в предметной области, увеличение понимания между заказчиком и командой разработки, посредством вырабатывания единого языка (Ubiquitous Language).

К недостаткам данного подхода относится необходимость высокой квалификации разработчиков для её эффективного применения, неприменимость без соответствующих архитектурных решений, что влечёт за собой увеличение затрат на первоначальное проектирование и увеличение итоговой стоимости проекта. По этим причинам DDD подходит проектам со сложной и долгоживущей бизнес-логикой, тогда как для небольших и краткосрочных проектов затраты на применение подхода, как правило, не окупаются.

Behavior-Driven Development (BDD) [45] – подход к разработке программного обеспечения, являющийся эволюцией метода Test-Driven Development. Основной принцип подхода состоит в том, что до написания какой-либо реализации специалисты предметной области совместно с разработчиками и тестировщиками описывают желаемое поведение системы на структурированном естественном языке. Наиболее распространённым форматом такого описания является конструкция Given-When-Then: предусловие (Given) задаёт начальное состояние системы, событие (When) описывает действие пользователя, а ожидаемый результат (Then) фиксирует наблюдаемое поведение, которое должна продемонстрировать система. Полученные сценарии преобразуются в автоматические тесты с помощью

специализированных инструментов, таких как Cucumber [1], SpecFlow [2] или Behave [3], и служат одновременно исполняемой спецификацией и приёмочными критериями.

Пример процесса разработки по BDD подходу показан на рисунке 6.

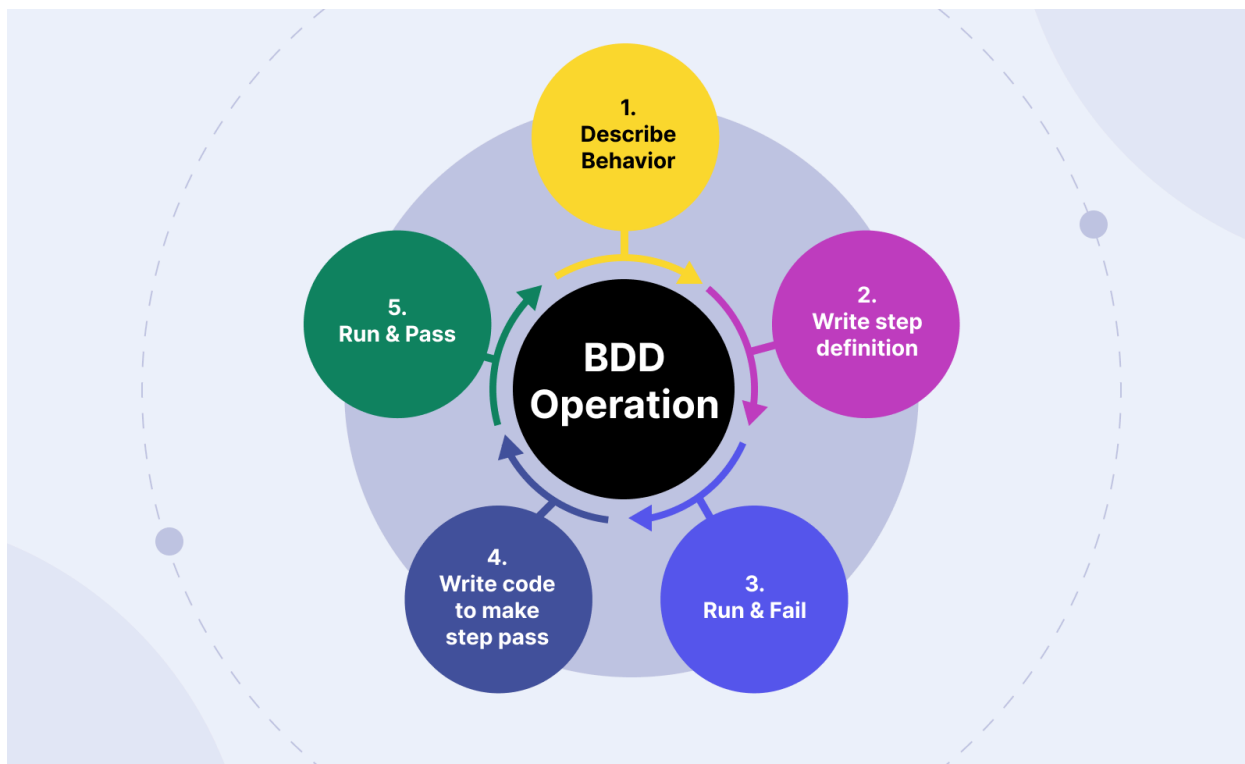


Рис. 6. Behavior-Driven Development

Подход решает ключевое ограничение TDD, при котором тесты остаются артефактом, понятным только разработчикам. В BDD сценарии становятся общим языком между заказчиком, аналитиком, тестировщиком и разработчиком, что снижает риск расхождения между реализованной функциональностью и бизнес-требованиями. Это достигается за счёт непосредственного участия представителей предметной области в формулировке сценариев на ранних стадиях разработки.

Вместе с тем применение BDD сопряжено с рядом издержек. Поддержание актуальности сценариев требует постоянных усилий при изменении требований. Внедрение подхода предполагает обучение всех

участников процесса работе с предметно-ориентированным языком и инструментами автоматизации. Совокупность этих факторов делает BDD более ресурсоёмким по сравнению с традиционными подходами, однако данные затраты, как правило, окупаются в проектах с высокой сложностью бизнес-логики и требованиями к долгосрочной сопровождаемости.

Spec-Driven Development (SDD) [40] – относительно новый подход, появившийся на фоне активного развития нейросетей и ИИ-агентов [43]. Суть подхода заключается в том, что перед написанием какой-либо реализации команда проекта определяет требования, поведение, технологии и ограничения программного обеспечения, фиксируя всё это в файлах спецификации. Данные файлы становятся источником истины как для человека, так и для ИИ-агента, что позволяет устранить ключевые системные ограничения при работе с языковыми моделями.

Выделяют четыре основные проблемы, решаемые данным подходом:

1. Ограниченность области и длительности задач: языковые модели демонстрируют деградацию качества при изменениях, затрагивающих более 3-5 файлов одновременно. SDD решает эту проблему посредством принудительной декомпозиции функциональности на атомарные задачи.

2. Отсутствие контекста о конкретной разрабатываемой функциональности: спецификации обеспечивают ИИ-агенту полное представление о требованиях, граничных случаях, критериях приёмки и точках интеграции ещё до начала генерации кода.

3. Незнание стандартов и соглашений конкретного проекта: специальный файл конституции (`constitution.md`) описывается

технологический стек, правила именования, архитектурные решения, допустимые библиотеки и требования безопасности.

4. Неконтролируемая автономия ИИ-агента: в рабочий процесс встраиваются обязательные контрольные точки проверки, на которых человек проверяет спецификацию, архитектурный план и декомпозицию задач перед запуском автоматической реализации.

Рабочий процесс SDD включает шесть последовательных этапов: формирование конституции проекта (Constitution), составление спецификации функциональности (Specify), устранение неоднозначностей (Clarify), архитектурное планирование (Plan), декомпозиция на задачи (Tasks) и непосредственная реализация (Implement). Такая структура смещает проектирование и принятие ключевых решений на ранние стадии разработки, сокращая объём переработок и технический долг на поздних этапах.

Для разработки backend-части информационной системы управления мероприятиями был выбран DDD подход, поскольку он упростит моделирование бизнес-логики управления мероприятиями, сделает программный код поддерживаемым для дальнейшего развития, упростит модульное тестирование, а также позволит приобрести необходимые в современных реалиях навыки проектирования, построения архитектуры и использования паттернов.

1.2. Архитектурные подходы backend разработки

При разработке backend-приложений, как и при разработке любого программного обеспечения, применяются различные архитектурные подходы

и паттерны, которые позволяют сделать код более читаемым, поддерживаемым, а также снизить порог входа для новых членов команды разработки.

Сперва рассмотрим архитектурные подходы на уровне приложения в целом.

Монолитная архитектура [36] – это подход к разработке программного обеспечения, при котором все модули тесно связаны между собой и представляют неделимое целое. В контексте клиент-серверных приложений монолитная архитектура предполагает размещение клиентского и серверного кода в рамках единого проекта, при этом клиентская часть неотделима от серверной (рис. 7).

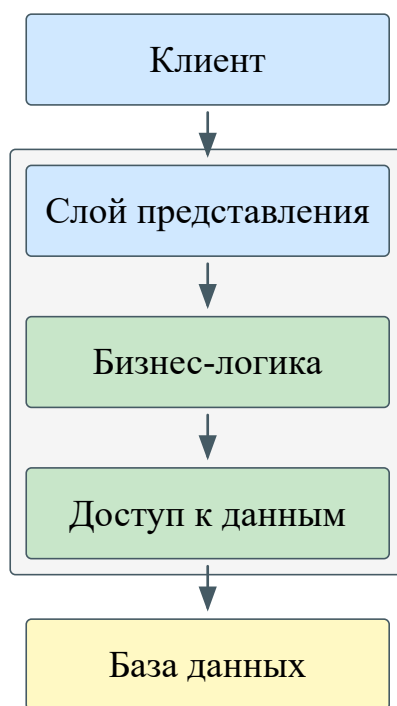


Рис. 7. Монолитная архитектура

К достоинствам монолитной архитектуры относятся простота и скорость разработки, обусловленные сосредоточением всей кодовой базы в одном месте, высокая производительность за счёт выполнения всех операций в рамках единого процесса, а также единая среда хранения данных, устраняющая необходимость в сложных механизмах их синхронизации.

Недостатки монолитной архитектуры обусловлены теми же характеристиками, что и её достоинства. В частности, масштабирование приложения осуществляется только как единого целого, что влечёт за собой избыточное потребление ресурсов. При росте числа пользователей исполнение всего кода в рамках одного процесса становится узким местом с точки зрения производительности. Помимо этого, внедрение новых технологий может потребовать перепроектирования всего приложения, что существенно замедляет его дальнейшее развитие.

На современном этапе монолитную архитектуру целесообразно рассматривать в качестве архитектурного подхода для небольших проектов с ограниченным жизненным циклом, корпоративных приложений с числом пользователей, не превышающим десятков или сотен человек, а также для проектов со сложной предметной областью.

Микросервисная архитектура [36] – это подход к разработке программного обеспечения, при котором программный код разделяется на отдельные автономные модули. В контексте клиент-серверных приложений микросервисная архитектура может рассматриваться на различных уровнях: отделение клиентской части от серверной, разделение клиентской части на микрофронтенды [44], а серверной – на микросервисы (рис. 8).

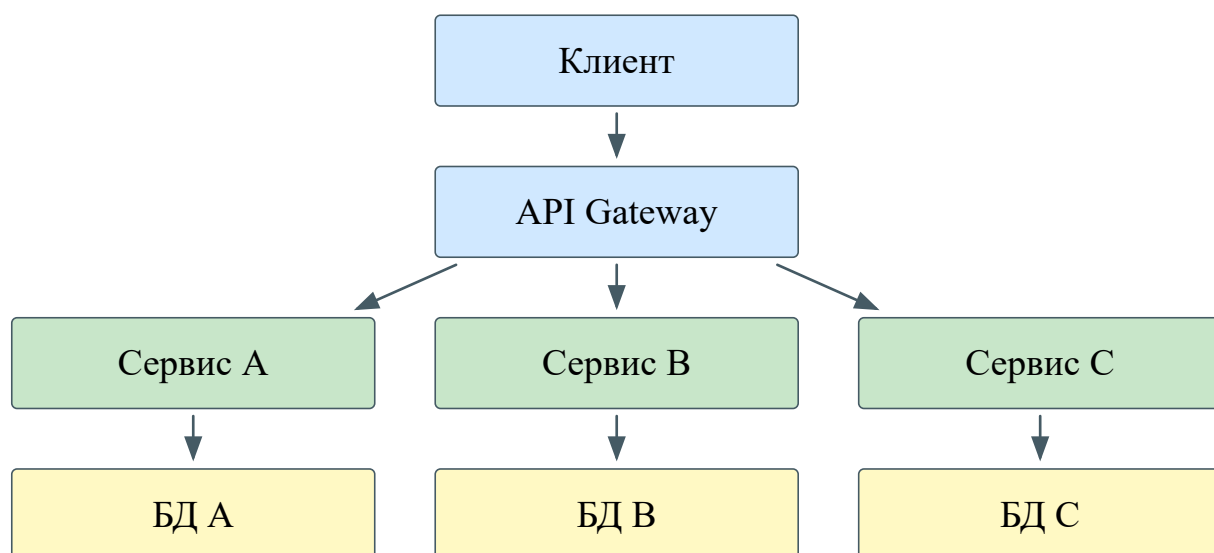


Рис. 8. Микросервисная архитектура

Достоинства микросервисной архитектуры непосредственно обусловлены её основополагающим принципом – разделением программного кода на автономные модули, что обеспечивает гибкость, масштабируемость, технологическое разнообразие и расширяемость системы.

К недостаткам микросервисной архитектуры относятся проблемы хранения данных и их синхронизация между модулями, для решения которых требуется применение дополнительных программных средств: очереди сообщений, таких как Apache Kafka [4] или RabbitMQ [5], веб-серверов и балансировщиков нагрузки, таких как nginx [6] или Apache HTTP Server [7], а также единого шлюза API (API Gateway), таких как Yandex API Gateway [8] или KrakenD [9]. Помимо этого, создание микросервисов предъявляет высокие требования к квалификации разработчиков, предполагая глубокие знания объектно-ориентированного программирования и практический опыт работы с перечисленными технологиями.

На современном этапе применение микросервисной архитектуры экономически обосновано преимущественно для крупных проектов с простой предметной областью, большим числом пользователей и потребностью

в постоянном масштабировании и расширении функциональности. Использование данного подхода характерно для организаций, располагающих собственным штатом разработчиков, в частности, для банков, таких как Альфа-Банк [10] или Т-Банк [11], а также маркетплейсов, например, Ozon [12] или Wildberries [13].

Модульно-монолитная архитектура [46] – это подход к разработке программного обеспечения, призванный сохранить достоинства монолитной архитектуры, такие как простота разработки и согласованность данных, дополнив их преимуществами микросервисной архитектуры: изоляцией модулей и технологическим разнообразием. Основопологающим принципом данного подхода является логическое разграничение модулей, в отличие от микросервисной архитектуры, где применяется физическое разграничение модулей (рис. 9).

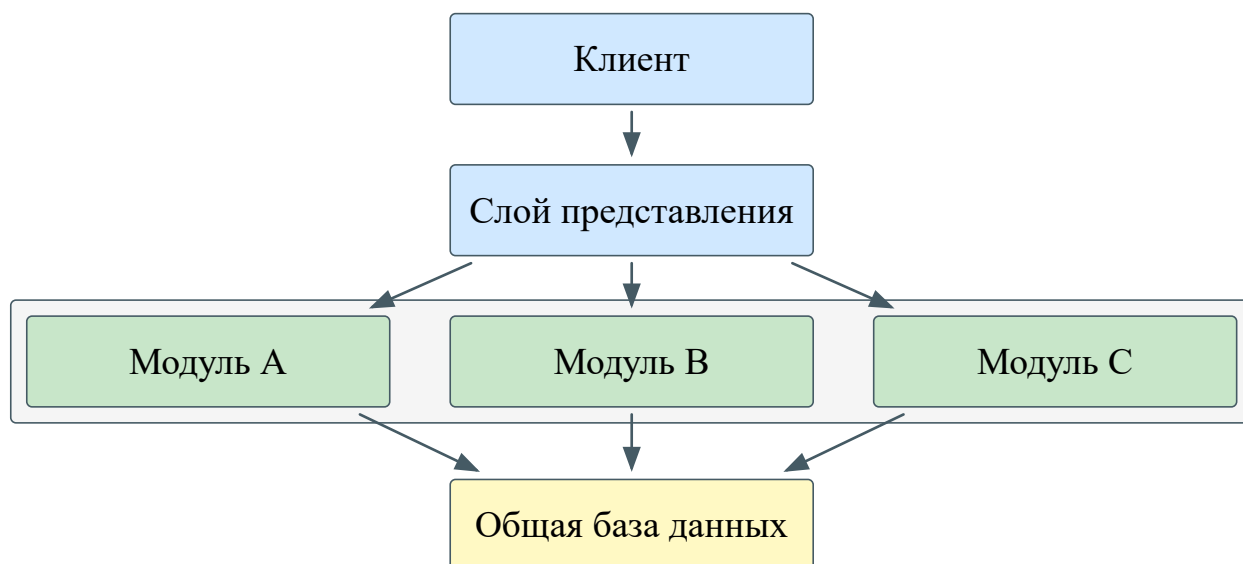


Рис. 9. Модульно-монолитная архитектура

Помимо достоинств, унаследованных от монолитной и микросервисной архитектур, модульный монолит обеспечивает возможность постепенного перехода от монолитной к микросервисной архитектуре а также разграничения

зон ответственности при расширении команды или проекта. Кроме того, данный подход снижает требования как к квалификации разработчиков, предъявляемые микросервисной архитектурой, так и к глубине знания предметной области, необходимой при монолитном подходе.

К недостаткам модульного монолита относится сохранение риска возникновения неявных межмодульных зависимостей при недостаточной дисциплине разработки, что способно привести к деградации архитектуры. Развёртывание приложения по-прежнему осуществляется целиком, что исключает возможность независимого обновления или масштабирования отдельных модулей. При значительном росте нагрузки модульный монолит сохраняет ограничения монолитной архитектуры в части горизонтального масштабирования. Поддержание чётких границ между модулями требует соблюдения архитектурных соглашений внутри команды, что повышает организационные издержки по мере роста проекта.

Рассмотренные архитектурные подходы отвечают на вопрос, как разделить систему на части. Однако вне зависимости от того, какой подход будет выбран, следующим уровнем архитектурных решений становится организация программного кода. Наиболее распространённые способы такой организации рассматриваются далее.

Слоистая архитектура [39] – подход, при котором программный код разделяется на горизонтальные слои, каждый из которых выполняет определённую роль в приложении.

Количество слоёв определяется индивидуально для каждого приложения, однако большинство слоистых архитектур включает четыре стандартных слоёв (рис. 10): слой представления (presentation layer),

отвечающий за отображение пользовательского интерфейса и данных, слой бизнес-логики (business layer), отвечающий за логику работы приложения, слой доступа к данным (persistence layer), отвечающий за получение и сохранение данных, и слой абстракции баз данных (database layer), отвечающий за подключение к базам данных. Последние два слоя могут быть объединены в один, особенно когда логика доступа к данным встроена в компоненты бизнес-логики или применяются технологии объектно-реляционного отображения (Object-Relational Mapping, ORM).

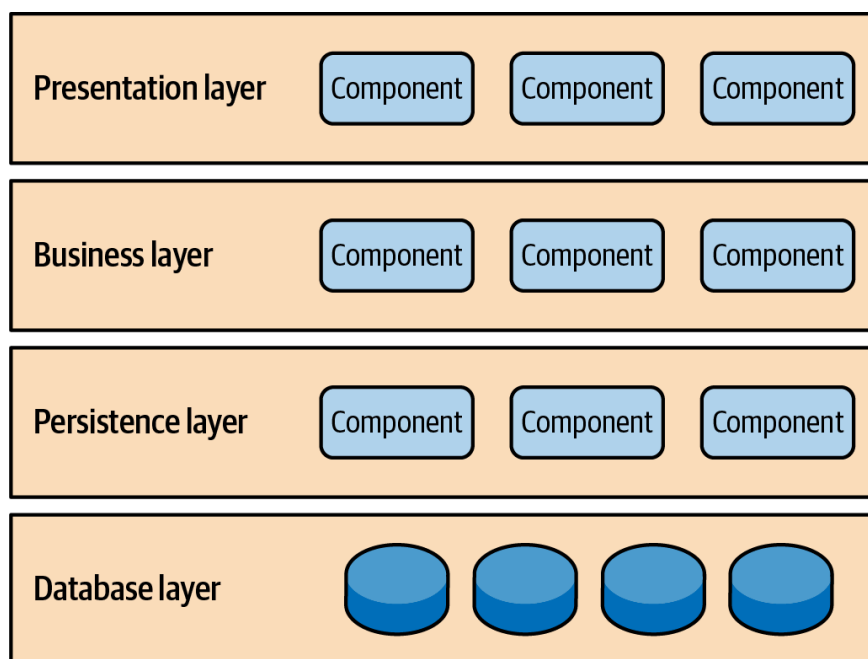


Рис. 10. Слоистая архитектура

Достоинством слоистой архитектуры является разделение ответственности между компонентами, позволяющее инкапсулировать логику каждого слоя и тем самым упростить разработку и тестирование. Ещё одним достоинством слоистой архитектуры выступает концепция закрытых слоёв (рис. 11): запрос, перемещаясь между слоями, обязан проходить через слой, непосредственно расположенный под ним, и не может миновать его.

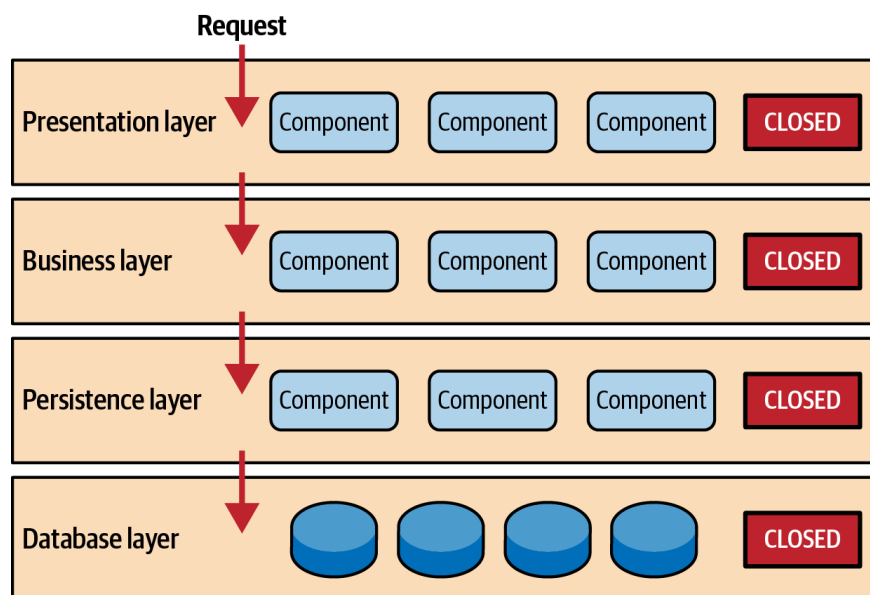


Рис. 11. Концепция закрытых слоёв в слоистой архитектуре

Применение концепции закрытых слоёв позволяет изолировать слои друг от друга, снизить связанность компонентов и вносить изменения в один слой, как правило, не затрагивая соседние.

К недостаткам слоистой архитектуры относят недостаточную изоляцию слоёв: архитектура не предполагает механизмов принудительного соблюдения слоёв, что допускает обращение через слой в обход установленного порядка. Кроме того, зависимость слоя бизнес-логики от слоя доступа к данным затрудняет замену одной СУБД на другую.

Луковая архитектура [38] – подход, при котором программный код организуется в виде концентрических слоёв, направление зависимостей в которых строго ориентировано к центру (рис. 12).

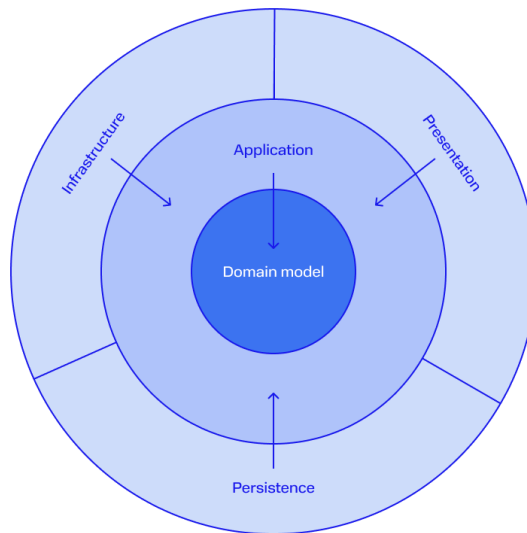


Рис. 12. Луковая архитектура

В центре архитектуры располагается слой предметной области (domain layer), содержащий бизнес-логику и бизнес-правила приложения. Поскольку все зависимости направлены внутрь, слой предметной области не зависит ни от одного из внешних слоёв и связан только с собственными компонентами.

Вокруг слоя предметной области располагается слой приложения (application layer), реализующий бизнес-сценарии (use-cases) и управляющий взаимодействием между слоем предметной области и внешними слоями посредством шлюзов (gateways). Шлюзы представляют собой интерфейсы, объявленные в ядре приложения и транслирующие запросы из модели предметной области во внешний мир и обратно. Конкретные реализации этих интерфейсов выносятся во внешние слои, что обеспечивает изоляцию ядра от деталей инфраструктуры.

Следующим слоем является слой инфраструктуры (infrastructure layer), обеспечивающий взаимодействие приложения с внешними зависимостями: базами данных, файловой системой, веб-сервисами и брокерами сообщений. На этом же уровне располагаются конкретные реализации интерфейсов,

объявленных во внутренних слоях. Наконец, внешним слоем является слой представления (presentation layer) – точка входа в приложение, реализующая публичный интерфейс взаимодействия: REST API, подписку на очередь сообщений и т.п.

Ключевым отличием луковой архитектуры от слоистой является отношение к базе данных. В луковой архитектуре база данных не является центром приложения, а выносится на внешний уровень в качестве сменяемой инфраструктурной зависимости. Такой подход смещает фокус проектирования с организации хранения данных на моделирование бизнес-процессов.

Достоинством луковой архитектуры является полная изоляция бизнес-логики от инфраструктурных зависимостей, что позволяет тестировать слой предметной области модульными тестами без необходимости использования базы данных или внешних сервисов. Кроме того, модульная структура упрощает замену инфраструктурных компонентов, например, переход с одной СУБД на другую, не затрагивая бизнес-логику.

К недостаткам подхода относят повышенную сложность проектирования по сравнению со слоистой архитектурой: разработчику необходимо корректно разграничивать ответственность слоёв и поддерживать строгое соблюдение правила зависимостей, что требует дисциплины и опыта.

Чистая архитектура [37] — подход, предложенный Робертом Мартином в 2012 году и описанный им в одноимённой книге. Чистая архитектура не является принципиально новым решением, а представляет собой синтез ранее предложенных идей: луковой архитектуры, гексагональной архитектуры и подхода Boundary - Control - Entity (BCE). Общей целью всех перечисленных

подходов является разделение ответственности посредством разбиения программного кода на слои (рис. 13).

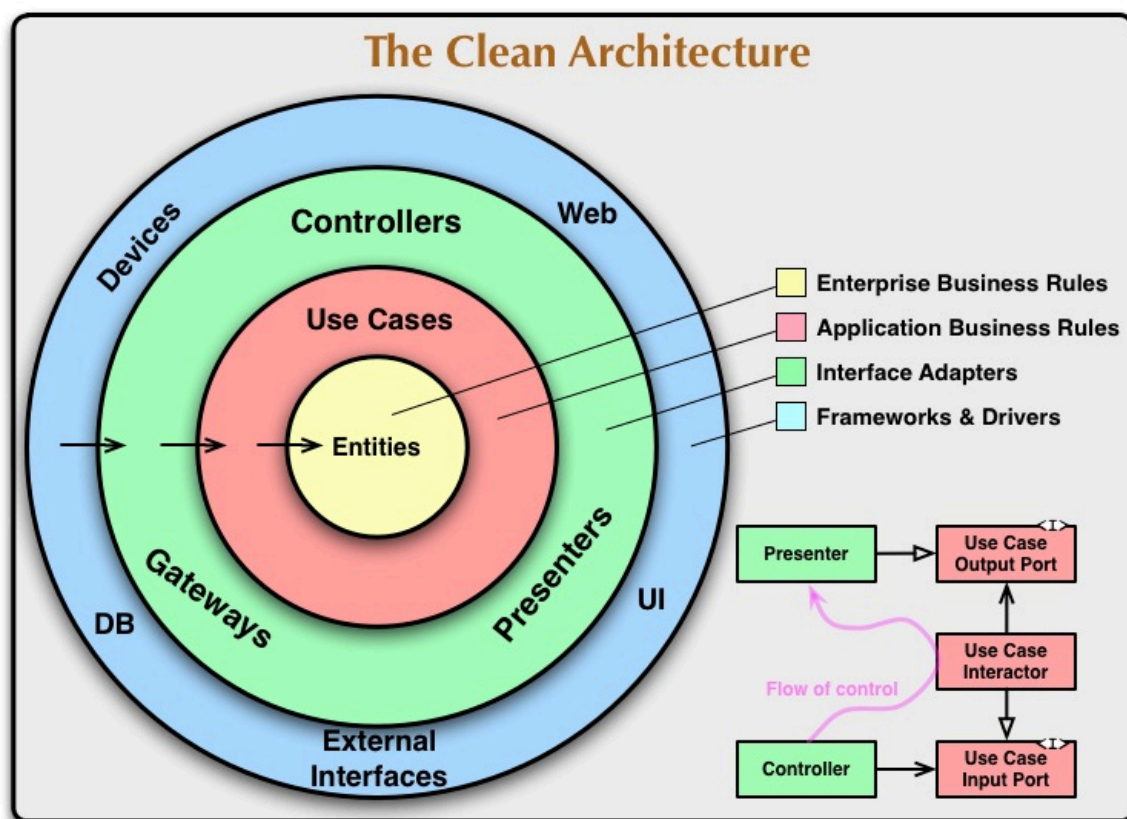


Рис. 13. Чистая архитектура

Центральным принципом чистой архитектуры является правило зависимостей (Dependency Rule): зависимости в исходном коде могут указывать только внутрь, в сторону слоёв с более высоким уровнем абстракции. Ни один компонент внутреннего слоя не должен содержать каких-либо сведений о компонентах внешних слоёв – включая имена классов, функций, переменных и форматы передаваемых данных.

Архитектура включает четыре концентрических слоя. В центре располагается слой сущностей (entities), инкапсулирующий корпоративные бизнес-правила – наиболее стабильную часть системы. Следующим слоем является слой вариантов использования (use cases), содержащий бизнес-правила конкретного приложения и управляющий потоком данных между

сущностями. Изменения в этом слое не затрагивают сущности, однако сам слой может изменяться при изменении бизнес-сценариев приложения. Далее следует слой адаптеров интерфейсов (interface adapters), преобразующий данные из формата, удобного для вариантов использования и сущностей, в формат, требуемый внешними системами – базами данных, веб-фреймворками и т.п. Внешним слоем является слой фреймворков и драйверов (frameworks and drivers), содержащий конкретные реализации инфраструктурных зависимостей, именно здесь сосредоточены все детали: веб-сервер, база данных, пользовательский интерфейс.

Ключевым отличием чистой архитектуры от луковой является более детальное разграничение внутренних слоёв: выделение сущностей и вариантов использования в два самостоятельных слоя позволяет явно разделить корпоративные правила, общие для нескольких приложений, и правила, специфичные для конкретного приложения. В луковой архитектуре это разграничение явно не регламентируется.

Достоинством чистой архитектуры является независимость бизнес-логики от фреймворков, баз данных и пользовательского интерфейса, что обеспечивает возможность полного покрытия бизнес-правил модульными тестами без привлечения инфраструктурных компонентов. Кроме того, изоляция внешних зависимостей упрощает их замену без изменения ядра системы.

К недостаткам подхода относят переусложнение архитектуры на начальном этапе разработки: строгое соблюдение правила зависимостей требует введения дополнительных интерфейсов и объектов передачи данных

(data transfer object, DTO) на границах каждого слоя, что увеличивает количество абстракций и усложняет кодовую базу в небольших проектах.

Для разработки backend-части информационной системы управления мероприятиями была выбрана модульно-монолитная архитектура. Данный выбор обусловлен требованием к встраиванию системы в существующую экосистему предприятия: модульная структура обеспечивает достаточную изолированность компонентов для их последующей интеграции со смежными системами, а при необходимости допускает декомпозицию отдельных модулей в самостоятельные микросервисы.

В качестве подхода к внутренней организации программного кода применяется чистая архитектура, обеспечивающая изоляцию бизнес-логики от инфраструктурных зависимостей и возможность независимого тестирования каждого слоя системы.

1.3. Анализ и выбор программных средств разработки

Разработка backend-приложений осуществляется на различных языках программирования с применением широкого спектра технологий, включающих фреймворки, системы контроля версий, инструменты контейнеризации, а также средства непрерывной сборки и развёртывания. Для обоснованного выбора технологического стека backend-части информационной системы управления мероприятиями далее рассматриваются наиболее распространённые из них.

В качестве языка программирования для разработки backend-приложений может выступать практически любой язык общего назначения, однако наибольшее распространение получили языки с устоявшейся

экосистемой библиотек и фреймворков, широким сообществом разработчиков и богатым набором инструментов. На современном этапе наиболее популярными языками программирования для backend-разработки являются Python, Java, C# и Go, каждый из которых рассматривается далее с точки зрения применимости в качестве основы для backend-приложения.

Python [14] – интерпретируемый язык программирования с динамической типизацией. К достоинствам языка относят лаконичный и логичный синтаксис, отсутствие необходимости явного приведения типов данных, богатый набор стандартных библиотек, а также широкий спектр сторонних библиотек и фреймворков для решения разнообразных задач. К недостаткам языка относят относительно невысокую производительность по сравнению с компилируемыми языками, а также сложность поддержания архитектурной целостности приложений вследствие особенностей языка, в частности отсутствия инкапсуляции членов класса. Наиболее распространёнными веб-фреймворками для данного языка являются Django [15], Flask [16] и FastAPI [17].

Java [18] – статически типизированный компилируемый язык программирования, исполняемый на виртуальной машине Java (Java Virtual Machine, JVM). К достоинствам языка относят строгую типизацию, позволяющую выявлять ошибки на этапе компиляции, высокую производительность, платформонезависимость, обусловленную исполнением в среде JVM, а также зрелую экосистему библиотек и инструментов, накопленную за десятилетия активного использования в корпоративной разработке. К недостаткам относят высокий порог вхождения, значительный объём шаблонного кода, а также повышенное потребление оперативной

памяти по сравнению с рядом других языков. Наиболее распространённым фреймворком для backend-разработки на Java является Spring Boot [19].

C# [20] – статически типизированный компилируемый язык программирования, разработанный компанией Microsoft и функционирующий в экосистеме платформы .NET. К достоинствам языка относят строгую типизацию, встроенное управление памятью посредством сборщика мусора, высокую производительность, кроссплатформенность, обеспечиваемую средой выполнения .NET Core. К недостаткам относят исторически сложившуюся зависимость от экосистемы Microsoft, что может ограничивать гибкость выбора инфраструктуры, а также необходимость освоения значительного числа концепций платформы .NET для эффективного использования языка. Основным фреймворком для backend-разработки на C# является ASP.NET Core [21].

Go [22] – статически типизированный компилируемый язык программирования, разработанный компанией Google. К достоинствам языка относят высокую производительность, обусловленную компиляцией в машинный код, встроенную поддержку конкурентного выполнения посредством горутин (goroutines), легковесных потоков исполнения, лаконичный синтаксис, а также низкое потребление ресурсов в сравнении с языками, исполняемыми на виртуальных машинах. К недостаткам языка относят относительно молодую экосистему с менее развитым, по сравнению с Java и C#, набором готовых решений для корпоративной разработки. Наиболее распространёнными веб-фреймворками для данного языка являются Gin [23] и Echo [24].

Для разработки backend-части проекта выбран язык программирования C# с применением фреймворка ASP.NET Core. Данный выбор обусловлен рядом факторов:

1. Платформа .NET обеспечивает встроенную поддержку механизмов, необходимых для реализации чистой архитектуры и подхода DDD: интерфейсов, внедрения зависимостей (Dependency Injection) и строгой типизации. В сравнении с Python, где перечисленные механизмы либо отсутствуют на уровне языка, либо реализуются через сторонние решения, платформа .NET предоставляет их в составе стандартной библиотеки.

2. Экосистема C# включает ORM-библиотеку Entity Framework Core [25], предоставляющую развитые средства работы с базой данных и снижающую объём шаблонного кода. В сравнении с Java и Go, где аналогичные инструменты требуют привлечения сторонних зависимостей с менее однородной документацией, данное решение обеспечивает более высокую согласованность технологического стека.

Выбор системы управления базами данных (СУБД) является одним из ключевых архитектурных решений при разработке backend-приложения. Далее рассматриваются наиболее распространённые реляционные СУБД – PostgreSQL и MySQL, а также Redis как представитель класса нереляционных хранилищ данных.

PostgreSQL [26] – объектно-реляционная система управления базами данных с открытым исходным кодом. К достоинствам данной СУБД относят полное соответствие стандарту ACID (Atomicity, Consistency, Isolation, Durability) [41], поддержку механизма многоверсионного управления конкурентным доступом (Multi-Version Concurrency Control, MVCC),

обеспечивающего высокую производительность при параллельных операциях чтения и записи, широкий набор поддерживаемых типов данных, включая JSON, геопространственные типы и пользовательские типы данных, а также развитую систему расширений. Кроме того, PostgreSQL распространяется под свободной лицензией, что исключает лицензионные издержки при его использовании. К недостаткам относят относительно высокую сложность начальной настройки и администрирования, а также повышенное потребление оперативной памяти по сравнению с рядом других СУБД.

MySQL [27] – реляционная система управления базами данных с открытым исходным кодом, разрабатываемая компанией Oracle Corporation. К достоинствам данной СУБД относят высокую производительность при операциях чтения, простоту установки и настройки, широкую распространённость и, как следствие, большое сообщество разработчиков и обширную документацию. К недостаткам MySQL относят ограниченную поддержку ряда возможностей стандарта SQL по сравнению с PostgreSQL, в частности оконных функций и полнотекстового поиска, сложность горизонтального масштабирования при высоких нагрузках, а также зависимость траектории развития продукта от решений коммерческого владельца, что снижает прозрачность процесса разработки.

Redis [28] – нереляционное хранилище данных типа «ключ – значение», функционирующее преимущественно в оперативной памяти. В отличие от реляционных СУБД, Redis не предназначен для хранения основного массива данных приложения, а применяется как дополнительный слой кэширования, хранилище пользовательских сессий, а также брокер сообщений с поддержкой паттерна публикации и подписки (Publish/Subscribe). К достоинствам Redis

относят исключительно высокую скорость операций чтения и записи, обусловленную хранением данных в памяти, поддержку разнообразных структур данных (строки, списки, множества, хэши, упорядоченные множества), а также наличие встроенных механизмов управления временем жизни записей (Time to Live, TTL). К недостаткам относят ограниченность объёма хранимых данных размером доступной оперативной памяти, риск потери данных при аварийном завершении работы, поскольку основным хранилищем является память, а не диск, а также отсутствие полноценной поддержки транзакций в объёме, сопоставимом с реляционными СУБД.

В качестве основной СУБД для разработки информационной системы управления мероприятиями выбран PostgreSQL. Данный выбор обусловлен необходимостью обеспечения целостности взаимосвязанных данных предметной области, развитой поддержкой типов данных, требующихся для хранения информации о мероприятиях, участниках и расписаниях, а также соответствием лицензионных условий требованиям встраивания системы в корпоративную экосистему без дополнительных издержек. Применение Redis рассматривается как перспективное направление развития системы для снижения нагрузки на основную базу данных посредством кэширования часто запрашиваемых данных и хранения пользовательских сессий.

Для хранения неструктурированных данных (файлов, изображений и вложений) в разрабатываемой системе применяется объектное хранилище, совместимое с протоколом Amazon S3 (Simple Storage Service). Протокол S3 является стандартом взаимодействия с объектными хранилищами, что обеспечивает наличие зрелых клиентских библиотек для различных языков программирования и платформ. Применение объектного хранилища позволяет

отделить логику хранения файлов от файловой системы сервера приложения, а также исключить хранение бинарных данных непосредственно в базе данных, что положительно влияет на её производительность. Далее рассматриваются два наиболее распространённых S3-совместимых хранилища для собственного развёртывания – MinIO и RustFS.

MinIO [29] – высокопроизводительное объектное хранилище с открытым исходным кодом, написанное на языке Go и обеспечивающее полную совместимость с API Amazon S3. К достоинствам MinIO относят зрелость продукта, широкую распространённость, детальную документацию и большое сообщество разработчиков. К недостаткам относят лицензионные ограничения: начиная с 2021 года MinIO распространяется под лицензией GNU Affero General Public License v3 (AGPLv3), которая при встраивании продукта в закрытую коммерческую систему обязывает раскрывать исходный код всей системы, либо приобретать коммерческую лицензию. Помимо этого, в 2025 году административная часть веб-интерфейса была перенесена из свободной редакции в коммерческую редакцию MinIO AIStor, что существенно сократило функциональность бесплатной версии продукта.

RustFS [30] – высокопроизводительное распределённое объектное хранилище с открытым исходным кодом, написанное на языке Rust и обеспечивающее полную совместимость с API Amazon S3. RustFS позиционируется как альтернатива MinIO: архитектурно оба продукта схожи, однако RustFS распространяется под лицензией Apache 2.0, не накладывающей ограничений на использование в закрытых коммерческих системах. К достоинствам RustFS относят свободную лицензию, реализацию на языке Rust, обеспечивающую безопасность работы с памятью и высокую

производительность при операциях с объектами малого размера, а также полную совместимость с инструментами и SDK, разработанными для Amazon S3. К недостаткам относят молодость проекта: на момент написания работы продукт находится в стадии альфа-тестирования, что предполагает возможное наличие нестабильностей и неполноту реализации ряда возможностей.

В качестве объектного хранилища для разрабатываемой системы выбран RustFS. Данный выбор обусловлен лицензионными ограничениями MinIO, несовместимыми с требованием встраивания системы в корпоративную экосистему без обязательного раскрытия исходного кода, тогда как лицензия Apache 2.0, применяемая в RustFS, допускает подобное использование без дополнительных условий.

Docker [31] – платформа контейнеризации с открытым исходным кодом, позволяющая упаковывать приложение вместе со всеми его зависимостями в изолированную единицу – контейнер. В отличие от виртуальных машин, контейнеры используют ядро операционной системы хост-машины совместно, не эмулируя отдельную операционную систему, что обеспечивает меньшее потребление ресурсов и более быстрое время запуска.

К достоинствам Docker относят воспроизводимость среды выполнения: контейнер, собранный на машине разработчика, гарантированно ведёт себя идентично в тестовой и производственной средах. К недостаткам относят сложность управления при значительном числе контейнеров, что требует привлечения дополнительных инструментов оркестрации.

Docker Compose [32] – инструмент для декларативного определения и запуска многоконтейнерных приложений, конфигурация которых описывается в едином файле формата YAML. К достоинствам относят централизацию

конфигурации сервисов и поддержку переменных окружения, позволяющую использовать единый файл конфигурации в различных средах. К недостаткам – ориентированность на однохостовые конфигурации и отсутствие встроенных средств балансировки нагрузки и автоматического восстановления после сбоев.

Для разрабатываемой системы выбраны Docker и Docker Compose в качестве инструментов контейнеризации и оркестрации. Данный выбор обусловлен требованием к воспроизводимости среды развёртывания для совместной работы, а также необходимостью совместного запуска нескольких компонентов системы – backend-приложения, базы данных PostgreSQL и объектного хранилища RustFS. Применение Kubernetes на данном этапе избыточно ввиду однохостового развёртывания системы без требований к горизонтальному масштабированию.

2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ

Мероприятие представляет собой организованное действие или совокупность действий, направленных на достижение определённой цели, предполагающее участие людей и ограниченное по времени и месту проведения.

В рамках разрабатываемой системы каждое мероприятие характеризуется следующими атрибутами: название, анонс, описание, дата проведения, а также тип и формат (офлайн, онлайн, гибрид). Для категоризации и поиска мероприятий предусматривается система тегов: каждое мероприятие может иметь один или несколько тегов. Мероприятия предполагают участие пользователей системы, пользователи, в свою очередь, имеют ФИО, адрес электронной почты и роль в системе.

Мероприятия проводятся в помещениях. Помещение характеризуется номером и типом (например: конференц-зал, переговорная), а также вместимостью. Помещения находятся в зданиях, которые имеют наименование и адрес. Для проведения мероприятий помещения оснащаются оборудованием. Каждая единица оборудования имеет наименование и относится к определённому типу оборудования.

2.1. Расчёт годового экономического эффекта

Для обоснования целесообразности разработки информационной системы управления мероприятиями необходимо оценить её экономическую эффективность. В качестве основного показателя выбран годовой экономический эффект (ГЭЭ), отражающий сокращение трудозатрат на

выполнение организационных операций в сравнении с существующим процессом. Расчёт выполнен на основе сравнительного анализа временных затрат на выполнение типовых операций организации мероприятий до и после внедрения системы. В качестве базового объекта расчёта принята организация с годовым объёмом мероприятий порядка 250 единиц, что соответствует среднему показателю крупного образовательного учреждения. Стоимость одной минуты рабочего времени специалиста рассчитана исходя из среднемесячного оклада в размере 28 000 рублей при стандартном рабочем времени и составляет 8 рублей в минуту.

Расчёт выполнен по формуле:

$$\mathcal{E} = N * (T_1 - T_2) * \text{ЗП_мин}$$

где N – количество операций в год, T_1 – время до внедрения (мин), T_2 – время после внедрения (мин), ЗП_мин – стоимость одной минуты рабочего времени специалиста (8 руб/мин, исходя из оклада ~28 000 руб/мес).

Количество мероприятий в год: ~250.

Таблица 1

Расчёт годовой экономии фонда заработной платы

№	Наименование операции	T_1 , мин	T_2 , мин	ΔT , мин	Экономия, руб.
1	Регистрация заявки в 1С	30	5	25	50 000
2	Согласование текста анонса	15	2	13	26 000
3	Публикация анонса на сайт ВУЗа	45	3	42	84 000
4	Публикация анонса на сайт института	40	3	37	74 000
5	Создание анонса в мессенджерах	20	5	15	30 000
6	Создание анонса в соц. сетях	25	5	20	40 000
7	Рассылка уведомлений участникам	10	1	9	18 000
8	Сбор обратной связи	12	2	10	20 000
Итого годовая экономия ФЗП:					342 000 руб.

После внедрения информационной системы возникают дополнительные затраты на её обслуживание:

Таблица 2

Расходы на администрирование

Показатель	Значение
ЗП администратора в час, руб.	1 300
Количество часов обслуживания в год, ч.	20
Рост нагрузки на администратора, руб.	26 000

Итоговый годовой экономический эффект определяется как разность между совокупной экономией фонда заработной платы, достигаемой за

счёт сокращения трудозатрат на выполнение организационных операций, и дополнительными расходами на администрирование внедрённой системы.

$$\text{ГЭЭ} = \text{Экономия ФЗП} - \text{Расходы на администрирование}$$

$$= 342000 - 26000 = 316000 \text{ руб.}$$

Таким образом, внедрение информационной системы управления мероприятиями обеспечивает годовой экономический эффект в размере 316 000 рублей. Полученный результат свидетельствует о высокой экономической целесообразности разработки: совокупная экономия фонда заработной платы более чем в 13 раз превышает сопутствующие затраты на администрирование системы, что подтверждает обоснованность предложенного решения и его практическую значимость для организаций с регулярным объёмом мероприятий.

2.2. Алгоритм текстового поиска

Поиск по текстовым данным является одной из ключевых функций информационных систем, работающих с контентом. При реализации поиска по словам среди мероприятий возникает необходимость учитывать морфологические особенности русского языка: одно и то же слово может встречаться в разных числовых, временных и падежных формах. Для решения этой проблемы существует два метода нормализации текста – лемматизация и стемминг [33].

Лемматизация – метод приведения слова к его словарной форме (лемме) с учётом грамматического контекста. Лемматизация опирается на морфологический анализ и словари языка, что обеспечивает высокую

точность нормализации. Например, слова «мероприятия», «мероприятие», «мероприятий» будут приведены к форме «мероприятие».

Стемминг — метод приведения слова к его корневой форме путём отсечения окончаний, суффиксов и приставок. В отличие от лемматизации, данный метод не прибегает к морфологическому анализу и словарям, что даёт более быстрый, но менее контекстный результат. Например, слова «мероприятия», «мероприятие», «мероприятий» будут приведены к форме «мероприяти». Метод используется поисковыми движками для более широкого охвата источников информации согласно введенному пользователем ключевому запросу.

Поскольку разрабатываемая система работает с русскоязычными массивами данных, но не требует высокой точности, а также учитывая, что зачастую пользователи используют для поиска 1-2 слова, что показало исследование Site Search 360 в 2018 году [34], в качестве метода нормализации был выбран стемминг.

Алгоритм будет реализован следующим образом. На вход подаётся поисковый запрос пользователя в виде строки, запрос подвергается токенизации, то есть извлечению слов, очищенных от знаков препинания и приведённых к нижнему регистру. Каждый токен подвергается стеммингу, в результате чего формируется множество нормализованных форм поискового запроса.

Параллельно из массива всех доступных мероприятий извлекаются текстовые поля: название, анонс и описание, а также массив тегов. Для каждого мероприятия аналогичным образом выполняется токенизация и стемминг

всех слов из указанных полей. Полученные множества стемов хранятся и используются для сопоставления.

На этапе сравнения для каждого мероприятия проверяется наличие пересечения между множеством стемов поискового запроса и множеством стемов мероприятия. Если хотя бы один стем из запроса присутствует в тексте мероприятия, оно включается в результирующую выборку. Итоговый список мероприятий возвращается пользователю в качестве результата поиска.

Такой подход позволяет находить релевантные мероприятия вне зависимости от того, в какой грамматической форме и насколько грамматически верно пользователь вводит поисковый запрос, что существенно повышает удобство и точность поиска.

2.3. Алгоритм рекомендаций

Системы рекомендаций позволяют предлагать пользователю контент, соответствующий его интересам, на основе анализа его предыдущего поведения. В рамках данной работы будет реализован алгоритм контентной фильтрации, основанный на статистическом методе TF-IDF, который позволяет выявлять ключевые слова, характеризующие интересы пользователя, и на их основе подбирать релевантные мероприятия.

Выбор статистического метода вместо использования нейросетей обоснован требованием к производительности системы, поскольку развёртывание и обучение нейросети потребует значительных финансовых вложений, в то время как TF-IDF не требует отдельных вычислительных мощностей и интеграции с существующим backend.

Рассмотрим алгоритм TF-IDF подробнее.

Term Frequency (TF, частота термина) – характеристика, показывающая, как часто данное слово встречается в массиве слов. Формула расчёта:

$$TF(t, d) = \frac{n_t}{\sum_k n_k},$$

где t – слово, d – массив слов, n_t – количество вхождений слова t в массив слов, $\sum_k n_k$ – общее количество слов в массиве.

Inverse Document Frequency (IDF, обратная частота документа) – характеристика, показывающая, насколько редко данное слово встречается во всей коллекции документов. Слова, присутствующие во всех документах, например, предлоги и союзы, получают низкий IDF и фактически исключаются из анализа как незначимые. Формула расчёта:

$$IDF(t, D) = \ln \frac{|D|}{|\{d_i \in D : t \in d_i\}|},$$

где t – слово, D – массивы слов, $|D|$ – количество массивов слов, $|\{d_i \in D : t \in d_i\}|$ – количество массивов слов, в которых содержится слово t (когда $n_t \neq 0$).

TF-IDF – итоговая метрика, вычисляемая как произведение TF и IDF. Она позволяет определить слова, которые одновременно часто встречаются в конкретном массиве слов и редко в остальных, то есть слова, наиболее характерные и содержательно значимые для данного текста. Формула расчёта:

$$TF\text{-}IDF(t, d, D) = TF(t, d) \cdot IDF(t, D)$$

Алгоритм рекомендаций будет реализован следующим образом. На первом этапе из базы данных извлекается список всех мероприятий, которые пользователь посетил ранее. Для каждого такого мероприятия объединяются текстовые поля: название, описание, анонс и теги – в корпус пользователя.

На втором этапе для корпуса вычисляется метрика TF-IDF по всем словам, предварительно подверженным стеммингу. По результатам вычислений отбираются пять слов с наибольшим суммарным значением TF-IDF по всему корпусу пользователя. Эти слова считаются ключевыми для его интересов.

На третьем этапе алгоритм перебирает все доступные актуальные мероприятия, которые пользователь ещё не посещал. Для каждого такого мероприятия проверяется наличие хотя бы одного из пяти ключевых слов в его корпусе. Мероприятия, содержащие ключевые слова, включаются в список рекомендаций и возвращаются пользователю.

Данный подход позволяет формировать персонализированные рекомендации без необходимости сбора данных о других пользователях и без применения коллаборативной фильтрации, что упрощает реализацию и снижает требования к объёму накопленных данных.

2.4. Проектирование базы данных

Проектирование базы данных является одним из ключевых этапов разработки backend-части информационной системы, поскольку структура хранимых данных непосредственно определяет корректность реализации бизнес-логики и производительность системы в целом.

В качестве инструмента моделирования применяется диаграмма «сущность – связь» (Entity-Relationship Diagram, ERD), позволяющая наглядно представить состав сущностей предметной области, их атрибуты и характер взаимосвязей между ними.

Модель разработана в соответствии с принципами нормализации реляционных баз данных и отражает структуру предметной области, выделенную в рамках применения подхода DDD: мероприятия, места проведения, пользователи, участники и сопутствующее оборудование (рис. 14).

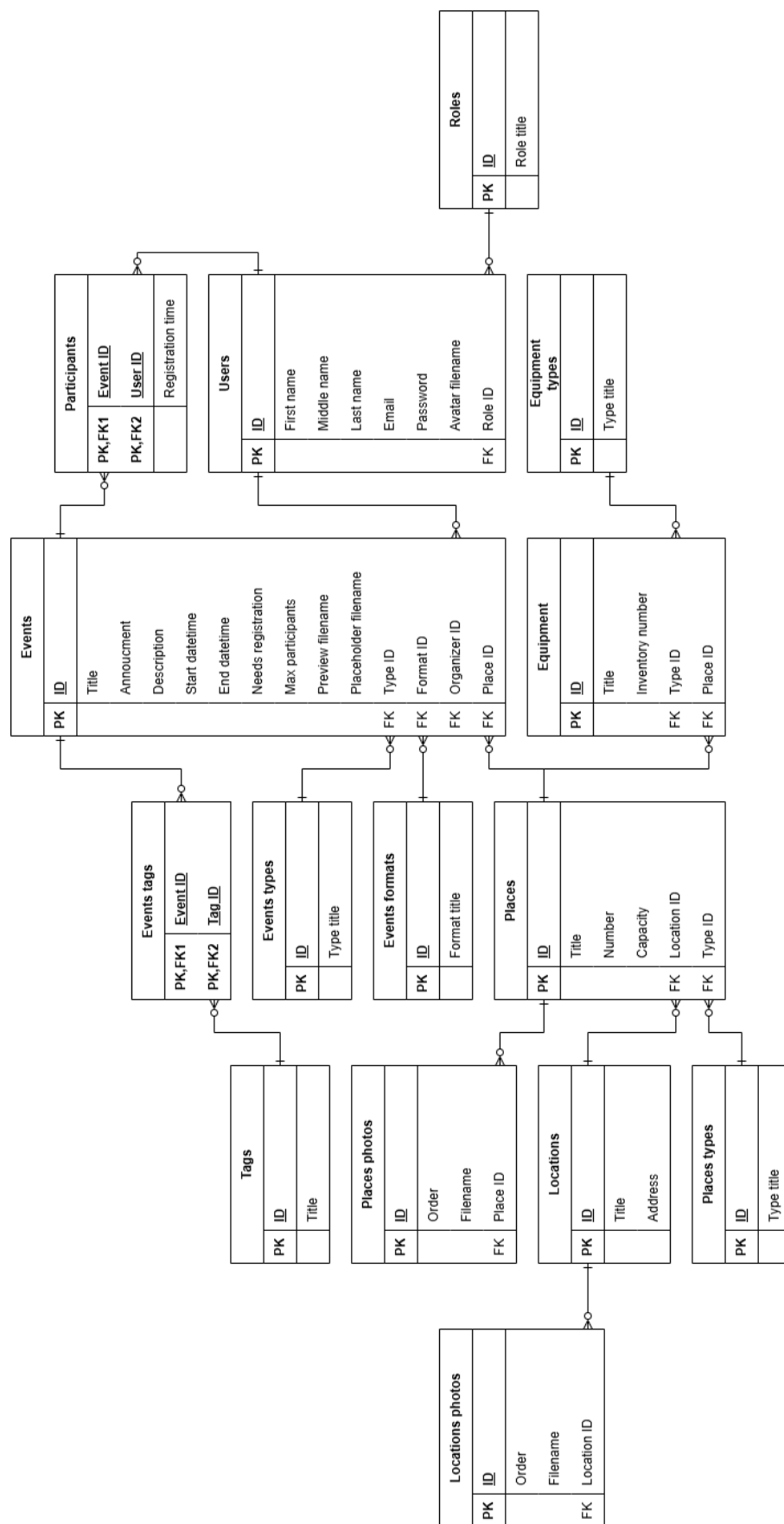


Рис. 14. Модель базы данных.

3. РАЗРАБОТКА BACKEND-ЧАСТИ СИСТЕМЫ УПРАВЛЕНИЯ МЕРОПРИЯТИЯМИ

3.1. Разработка доменного слоя

В рамках разработки доменного слоя по принципам DDD были определены два основных абстрактных класса – для сущностей и объектов-значений.

Для класса сущности было реализовано поле идентификатора, по которому осуществляется сравнение экземпляров, обеспечена поддержка различных типов данных для идентификаторов (например, int, Guid или пользовательский класс), переопределены механизмы сравнения двух сущностей.

```
/// <summary>
///     Абстрактный класс сущности.
/// </summary>
/// <typeparam name="TKey">Тип первичного ключа.</typeparam>
public abstract class Entity<TKey> : IEquatable<Entity<TKey>>
    where TKey : IEquatable<TKey>
{
    /// <summary>
    ///     Для EF Core.
    /// </summary>
    protected Entity()
    {
    }

    /// <summary>
    ///     Конструктор сущности.
```

```

/// </summary>
/// <param name="id">Идентификатор сущности.</param>
protected Entity(TKey id)
{
    Id = id;
}

/// <summary>
///     Идентификатор сущности.
/// </summary>
public TKey Id { get; }

public bool Equals(Entity<TKey>? other)
{
    if (other is null)
        return false;

    if (ReferenceEquals(this, other))
        return true;

    if (GetType() != other.GetType())
        return false;

    if (IsTransient() || other.IsTransient())
        return false;

    return Id.Equals(other.Id);
}

```

```

    /// <inheritdoc />
    public override bool Equals(object? obj)
    {
        return obj is Entity<TKey> entity && Equals(entity);
    }

    /// <inheritdoc />
    public override int GetHashCode()
    {
        if (IsTransient())
            return base.GetHashCode();

        return Id.GetHashCode();
    }

    /// <summary>
    ///     Оператор проверки равенства сущностей.
    /// </summary>
    /// <param name="left">Левый операнд.</param>
    /// <param name="right">Правый операнд.</param>
    /// <returns>
    ///     True, если сущности равны, false иначе.
    /// </returns>
    public static bool operator ==(Entity<TKey>? left,
Entity<TKey>? right)
    {
        return left?.Equals(right) ?? right is null;
    }

```

```

    /// <summary>
    ///     Оператор проверки неравенства сущностей.
    /// </summary>
    /// <param name="left">Левый операнд.</param>
    /// <param name="right">Правый операнд.</param>
    /// <returns>
    ///     True, если сущности неравны, false иначе.
    /// </returns>
    public static bool operator !=(Entity<TKey>? left,
Entity<TKey>? right)
    {
        return !(left == right);
    }

    /// <summary>
    ///     Проверка на присвоение ID.
    /// </summary>
    /// <returns>True, если ID не присвоен (равен значению по
умолчанию), false если ID присвоен.</returns>
    public bool IsTransient()
    {
        return EqualityComparer<TKey>.Default.Equals(Id, default);
    }
}

```

Для класса объекта-значения был реализован механизм сравнения на основе абстрактного метода `GetEqualityComponents`, возвращающего набор полей через `yield`. Каждый наследующий класс обязан переопределить данный

метод, указав поля, участвующие в сравнении. Хэш-код объекта также вычисляется на основе этих компонентов с использованием GetHashCode.

```
/// <summary>
///     Абстрактный класс объекта-значения.
/// </summary>
public abstract class ValueObject : IEquatable<ValueObject>
{
    /// <inheritdoc />
    public bool Equals(ValueObject? other)
    {
        if (other is null)
            return false;

        if (ReferenceEquals(this, other))
            return true;

        if (GetType() != other.GetType())
            return false;

        return
            GetEqualityComponents().SequenceEqual(other.GetEqualityComponents());
    }

    /// <summary>
    ///     Метод получения параметров для сравнения.
    /// </summary>
    /// <returns>
    ///     Объекты для сравнения через yield.
    /// </returns>
```

```

        protected abstract IEnumerable<object?>
GetEqualityComponents();

    /// <inheritdoc />
    public override bool Equals(object? obj)
    {
        return obj is ValueObject valueObject &&
Equals(valueObject);
    }

    /// <inheritdoc />
    public override int GetHashCode()
    {
        var hash = new HashCode();

        foreach (var component in GetEqualityComponents())
            hash.Add(component);

        return hash.ToHashCode();
    }

    /// <summary>
    ///     Оператор проверки равенства объектов.
    /// </summary>
    /// <param name="left">Левый операнд.</param>
    /// <param name="right">Правый операнд.</param>
    /// <returns>
    ///     True, если объекты равны, false иначе.
    /// </returns>

```

```

        public static bool operator ==(ValueObject? left, ValueObject?
right)
        {
            return left?.Equals(right) ?? right is null;
        }

        /// <summary>
        ///     Оператор проверки неравенства объектов.
        /// </summary>
        /// <param name="left">Левый операнд.</param>
        /// <param name="right">Правый операнд.</param>
        /// <returns>
        ///     True, если объекты неравны, false иначе.
        /// </returns>
        public static bool operator !=(ValueObject? left, ValueObject?
right)
        {
            return !(left == right);
        }
    }

```

Оба класса реализуют интерфейс `IEquatable`, что обеспечивает возможность использования в качестве типа идентификатора сущности как простых типов, так и пользовательских классов. Важным отличием проверки идентичности сущностей от объектов-значений является то, что для сущности выполняется сравнение только идентификаторов, тогда как для объектов-значений необходимо сравнение всех полей.

Предложенная реализация обеспечивает гибкость при описании сущностей и объектов-значений в рамках доменной модели.

Поскольку объекты-значения являются составными блоками сущностей, в качестве примера рассмотрим реализацию класса для текстовых полей.

```
/// <summary>
///     Базовый класс для текстовых полей.
/// </summary>
public class TextField : ValueObject
{
    /// <summary>
    ///     Для EF Core.
    /// </summary>
    private TextField()
    {
    }

    public TextField(
        string value,
        int maxLength,
        int minLength = 1,
        Error? nullError = null,
        Error? tooShortError = null,
        Error? tooLongError = null)
    {
        if (string.IsNullOrEmpty(value))
            throw new DomainException(nullError ??
TextFieldErrors.ValueIsNull);

        var trimmed = value.Trim();

        if (trimmed.Length < minLength)
```

```

        throw new DomainException(tooShortError ??
TextFieldErrors.ValueTooShort(minLength));

        if (trimmed.Length > maxLength)
            throw new DomainException(tooLongError ??
TextFieldErrors.ValueTooLong(maxLength));

        Value = trimmed;
    }

    /// <summary>
    ///     Значение текстового поля.
    /// </summary>
    public string Value { get; } = null!;

    protected override IEnumerable<object?> GetEqualityComponents()
    {
        yield return Value;
    }
}

```

Класс `TextField` наследуется от `ValueObject` и переопределяет метод `GetEqualityComponents`, возвращая единственное поле `Value`. Конструктор класса принимает строку и параметры для валидации – максимальную и минимальную длину, а также необязательные объекты ошибок для различных случаев некорректного значения. При создании экземпляра `TextField` выполняется проверка на `null` или пустую строку, а также на соответствие заданным ограничениям по длине, что обеспечивает инвариант класса и гарантирует корректность данных. Класс `TextField` может использоваться

в качестве типа для текстовых полей сущностей, обеспечивая при этом встроенную валидацию и поддержку сравнения на основе значения.

Пример использования `TextField` в качестве базового класса для названия мероприятия представлен ниже.

```
/// <summary>
///     Название мероприятия.
/// </summary>
public class Title : TextField
{
    public const int MaxLength = 128;

    public Title(string value) : base(
        value,
        MaxLength,
        nullError: EventTitleErrors.NullOrWhitespace,
        tooLongError: EventTitleErrors.GreaterThanMaxLength)
    {
    }
}
```

Наследование от класса `TextField` позволяет инкапсулировать в наследниках только необходимую логику, исключая дублирование базовых проверок. Это позволяет сущностям, составленным из подобных полей, быть сфокусированными на оркестрации бизнес-правил, а не на валидации данных.

3.2. Разработка инфраструктурного слоя

3.3. Разработка слоя бизнес-логики

3.4. Разработка слоя API

ЗАКЛЮЧЕНИЕ

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Официальная документация Cucumber [Электронный ресурс]. URL: <https://cucumber.io/> (дата обращения: 15.02.2026).
2. Официальная документация SpecFlow [Электронный ресурс]. URL: <https://www.specflow.com/> (дата обращения: 15.02.2026).
3. Официальная документация фреймворка Behave [Электронный ресурс]. URL: <https://behave.readthedocs.io/en/latest/> (дата обращения: 15.02.2026).
4. Официальная документация Apache Kafka [Электронный ресурс]. URL: <https://kafka.apache.org/> (дата обращения: 27.02.2026).
5. Официальная документация RabbitMQ [Электронный ресурс]. URL: <https://www.rabbitmq.com/> (дата обращения: 27.02.2026).
6. Официальная документация nginx [Электронный ресурс]. URL: <https://nginx.org/> (дата обращения: 27.02.2026).
7. Официальная документация Apache HTTP Server Project [Электронный ресурс]. URL: <https://httpd.apache.org/> (дата обращения: 27.02.2026).
8. Официальная документация Yandex API Gateway [Электронный ресурс]. URL: <https://yandex.cloud/ru/services/api-gateway> (дата обращения: 27.02.2026).
9. Официальная документация KrakenD [Электронный ресурс]. URL: <https://www.krakend.io/> (дата обращения: 27.02.2026).
10. Альфа-Банк. Технологический радар [Электронный ресурс]. URL: <https://digital.alfabank.ru/techradar/> (дата обращения: 27.02.2026).
11. Т-Банк. Работа в ИТ [Электронный ресурс]. URL: <https://www.tbank.ru/career/it/about/> (дата обращения: 27.02.2026).
12. Ozon Tech. Технологический стек [Электронный ресурс]. URL: <https://ozon.tech/> (дата обращения: 27.02.2026).

13. WBTECH. Технологический фундамент Wildberries [Электронный ресурс]. URL: <https://wbtech.wildberries.ru/> (дата обращения: 27.02.2026).
14. Официальная документация языка Python [Электронный ресурс]. URL: <https://www.python.org/> (дата обращения: 02.03.2026).
15. Официальная документация фреймворка Django [Электронный ресурс]. URL: <https://www.djangoproject.com/> (дата обращения: 02.03.2026).
16. Официальная документация фреймворка Flask [Электронный ресурс]. URL: <https://flask.palletsprojects.com/en/stable/> (дата обращения: 02.03.2026).
17. Официальная документация фреймворка FastAPI [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/> (дата обращения: 02.03.2026).
18. Официальная документация языка Java [Электронный ресурс]. URL: <https://www.java.com/ru/> (дата обращения: 02.03.2026).
19. Официальная документация фреймворка Spring Boot [Электронный ресурс]. URL: <https://spring.io/projects/spring-boot/> (дата обращения: 02.03.2026).
20. Официальная документация языка C# [Электронный ресурс]. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/> (дата обращения: 02.03.2026).
21. Официальная документация ASP.NET Core [Электронный ресурс]. URL: <https://dotnet.microsoft.com/ru-ru/apps/aspnet/> (дата обращения: 02.03.2026).
22. Официальная документация языка Go [Электронный ресурс]. URL: <https://go.dev/> (дата обращения: 02.03.2026).
23. Официальная документация фреймворка Gin [Электронный ресурс]. URL: <https://gin-gonic.com/> (дата обращения: 02.03.2026).
24. Официальная документация фреймворка Echo [Электронный ресурс]. URL: <https://echo.labstack.com/> (дата обращения: 02.03.2026).

25. Официальная документация Entity Framework Core [Электронный ресурс]. URL: <https://learn.microsoft.com/en-us/ef/core/> (дата обращения: 05.03.2026).
26. Официальная документация PostgreSQL [Электронный ресурс]. URL: <https://www.postgresql.org/> (дата обращения: 05.03.2026).
27. Официальная документация MySQL [Электронный ресурс]. URL: <https://www.mysql.com/> (дата обращения: 05.03.2026).
28. Официальная документация Redis [Электронный ресурс]. URL: <https://redis.io/> (дата обращения: 05.03.2026).
29. Официальная документация MinIO [Электронный ресурс]. URL: <https://www.min.io/> (дата обращения: 05.03.2026).
30. Официальная документация RustFS [Электронный ресурс]. URL: <https://rustfs.com/> (дата обращения: 05.03.2026).
31. Официальная документация Docker [Электронный ресурс]. URL: <https://www.docker.com/> (дата обращения: 06.03.2026).
32. Официальная документация Docker Compose [Электронный ресурс]. URL: <https://docs.docker.com/compose/> (дата обращения: 06.03.2026).
33. Stemming, Lemmatization - Which One is Worth Going For? [Электронный ресурс]. URL: <https://towardsdatascience.com/stemming-lemmatization-which-one-is-worth-going-for-77e6ec01ad9c/> (дата обращения: 18.02.2026).
34. What we learned from 10 million website search tool enquiries [Электронный ресурс]. URL: <https://www.sitesearch360.com/blog/what-we-learned-from-10-million-website-search-tool-enquiries/> (дата обращения: 18.02.2026).
35. Avram A., Marinescu F. Domain Driven Design Quickly / Avram A., Marinescu F., C4Media, 2006.

36. Doglio F. Monolith vs Microservice Architecture: A Comparison [Электронный ресурс]. URL: <https://camunda.com/blog/2023/08/monolith-vs-microservice-architecture-comparison/> (дата обращения: 26.02.2026).
37. Martin R. C. The Clean Architecture [Электронный ресурс]. URL: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> (дата обращения: 01.03.2026).
38. Palermo J. Onion Architecture [Электронный ресурс]. URL: <https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/> (дата обращения: 01.03.2026).
39. Richards M. Software Architecture Patterns Second Edition / Richards M., O'Reilly Media, Inc., 2022.
40. Shandrokha S. [и др.]. Inside Spec-Driven Development: What GitHub's Spec Kit Makes Possible For AI-assisted Engineering [Электронный ресурс]. URL: <https://www.epam.com/insights/ai/blogs/inside-spec-driven-development-what-githubspec-kit-makes-possible-for-ai-engineering> (дата обращения: 18.02.2026).
41. Sheldon R. <https://www.techtarget.com/searchdatamanagement/definition/ACID> [Электронный ресурс]. URL: <https://www.techtarget.com/searchdatamanagement/definition/ACID> (дата обращения: 05.03.2026).
42. Беспясов С. Разработка через тестирование (TDD) [Электронный ресурс]. URL: <https://doka.guide/tools/tdd/> (дата обращения: 15.02.2026).
43. Суворова М. AI-агенты: как они устроены и что умеют [Электронный ресурс]. URL: <https://cloud.ru/blog/kak-ustroyeny-i-chto-umeuyut-ai-agenty> (дата обращения: 18.02.2026).

44. Тульцева К. Что такое микрофронтенд [Электронный ресурс]. URL: <https://thecode.media/chto-takoe-mikrofrontend/> (дата обращения: 27.02.2026).
45. Фергюсон С. Д., Молак Я. BDD в действии / Фергюсон С. Д., Молак Я., ДМК-Пресс, 2023.
46. Цветчих Д. Модульный монолит вместо микросервисов [Электронный ресурс]. URL: https://www.youtube.com/watch?v=4UKjtzeQ_uc (дата обращения: 27.02.2026).

ПРИЛОЖЕНИЕ

Репозиторий – <https://github.com/Shtskkh/Events.Backend>

Выпускная квалификационная работа выполнена мной совершенно самостоятельно. Все использованные в работе материалы и концепции из опубликованной научной литературы и других источников имеют ссылки на них.

« ____ » _____ 2026 г.

(подпись)

(_____)
(Ф.И.О)